Defending Attack Graphs That Move: Telemetry-Driven Exposure, Incremental Shortest Paths, and Joint Detection–Deception Placement

Internal draft v1 — not for distribution

Lucas Sotomayor
BASE4 Security, R&D+i

November 4, 2026

Abstract

Attack-graph defenses for Active Directory (AD) are computed on snapshots, yet the edges that matter most—interactive sessions, local-admin grants, ACL changes—appear and expire within hours. We treat the attack graph as a time-indexed state G_t maintained from security telemetry and build four components on it. (i) A deterministic, idempotent *projection rule* from OCSF provenance events to attack-graph edge changes, implemented in Python and in the production engine (Rust) against one 17-case conformance suite; writing it exposed two defects in a production parser. (ii) An *incremental oracle* for crown-jewel distances with a work-based fallback: on a synthetic AD graph with 4.6k nodes and 354k edges, sessions and admin grants are repaired in 0.02–0.16 ms on average against 310 ms for recomputation, and the worst case stays within 1.1× of recomputation. (iii) A *joint detection–deception* model, obtained by a node-splitting reduction that makes traps ordinary rules for an exact per-path-guarantee MILP; with two traps, 18 of 20 synthetic instances admit a $\delta = 0.05$ guarantee (6 with rules alone) at a median 11% of the rules-only cost. (iv) A *combined defender* that minimises the value collected by a second attacker model, which gathers loot node by node, subject to the first model's guarantee. Cross-evaluating defenses against both attackers shows that each single-model optimum fails the other: loot-optimal traps give the path guarantee in 4 of 15 (instance, budget) pairs, and guarantee-optimal traps leave about half the loot. The combined defender gives the guarantee in every feasible case at a median loot within 1% of the loot-optimal placement for budgets of two and three traps. (v) A *joint defender* that chooses rules and traps together by outer approximation, exploiting that the attacker's value is convex in the rule choice; trading alert load against a loot cap, two traps reach half the undefended loot with no rules in all five instances, and a 10% cap costs a median 23% fewer false positives per day than rules alone. All results use synthetic detection content and synthetic or small AD graphs; every result is checked against brute force or full recomputation.

1 Introduction

Defensive analyses of AD attack graphs, from BloodHound path queries [24] to edge blocking [5, 6] and honeypot placement [14], take a collection snapshot as input. Two observations motivate this work. First, the edges that create short paths to Domain Admin are largely *transient*: a privileged user logging on to a workstation creates a `HasSession` edge that lasts as long as the credentials stay in memory. A snapshot taken on Monday says little about Thursday. Second, the telemetry that reveals those edges is already collected by the SOC—Windows 4624/4688, Sentinel, EDR—but it lives in a different graph: a provenance graph of what happened, not an attack graph of what could happen.

We connect the two. The attack graph becomes a state G_t updated by events, and every defensive question becomes a query or an optimisation over G_t.

Contributions.

- A two-layer model of G_t and a **projection rule** from OCSF provenance events to attack-graph changes, with explicit edge support, idempotent deltas, watermark-based expiry and identity resolution that refuses to guess (Section 4). A reference implementation passes a 17-case conformance suite shared with the production engine.
- An **incremental distance oracle** for crown-jewel exposure in the style of Ramalingam–Reps [20], with a fallback triggered by the *edge work* of the affected subtree rather than its size, which matters with group hubs (Section 5).
- A **joint detection–deception** formulation: a reduction that turns node traps into edge rules, so that one exact MILP chooses detection rules and traps under a per-path guarantee, plus a trap budget (Section 6).

- A **combined defender** for two attacker models on the same graph: traps that minimise a value-collecting attacker's loot subject to the path guarantee, by column generation over a master that joins both sets of constraints (Section 6.1).
- A **joint defender** that also chooses the rules, by outer approximation with tangent cuts on a convex value function, and the resulting trade-off between alert load and loot (Section 6.2).
- An evaluation on GOADv2 [18, 11] and synthetic AD graphs, including a cross-evaluation of every defense against both attackers (Section 7), with every result verified against an independent reference (brute force or full Dijkstra).

2 Background and related work

Attack graphs and hardening. Attack graphs were formalised by Jha et al. [9]; minimum-cost hardening [26] and IDS sensor placement [16] are early optimisation uses. For AD, Guo et al. study edge blocking with fixed-parameter [5] and scalable algorithms [6], and Goel et al. combine a neural dynamic program with evolutionary search [4].

Interdiction and security games. Shortest-path network interdiction [7] and its stochastic variants [13] are the operations-research root of our defender. Double-oracle methods solve zero-sum security games on graphs [8]. Alert triage as a game is studied by Schlenker et al. [21]. Nguyen, Song and Smith combine arc removal with a simultaneous monitoring game against an evader [15]; in our setting monitoring is a shared rule that covers many edges at once, the objective is alert load under a per-path guarantee, and traps and a value-collecting attacker are added.

Deception. Honeypot allocation over attack graphs [1], the survey by Pawlick et al. [19], and honeypot placement in dynamic AD graphs [14] are closest to our trap model. Ngo et al. also treat the graph as dynamic, but the dynamics are a model of the attacker's moves, not a stream of observed telemetry.

Dynamic shortest paths. Incremental single-source shortest paths go back to Ramalingam and Reps [20]; LPA* [10] is the planning variant; Demetrescu and Italiano [3] give fully dynamic all-pairs bounds. We use the simplest of these ideas and show where it pays off on AD graphs and where it does not.

Telemetry schemas. OCSF [17] is the event schema of the projection; detection content is mapped to ATT&CK techniques, whose use in EDR products was measured by Virkud et al. [25].

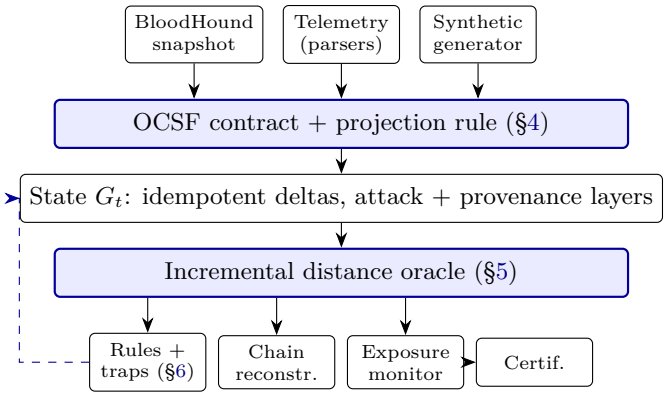


Figure 1: Architecture. Shaded components are new in this work. The dashed arrow is the feedback loop: what the defender deploys enters G_t as deltas.

3 The state G_t

Definition 1 (Two-layer state)**.** $G_t = (V_t,\ E_t^{\text{atk}} \cup E_t^{\text{prov}},\ w_t,\ q_t,\ r)$. *Attack edges* $(u, \kappa, v) \in E_t^{\text{atk}}$ *mean "controlling* u *lets the attacker acquire* v*" with BloodHound semantics and directions [24]:* `HasSession` *goes from host to user,* `AdminTo` *and* `CanRDP` *from principal to host. Provenance edges are timestamped OCSF events.* q_t *are detection probabilities on edges (rules) or nodes (traps),* w_t *are attack-edge weights and* r *node values.*

Each attack edge carries its *origins* (`snapshot`, `observed`), its *support* (the set of event ids sustaining it), first and last evidence times, an expiry time and a confidence (`confirmed` if an event shows it exercised, `inferred` if it is only enabled). An edge exists in G_t while it has the snapshot origin, or a non-empty support and an unexpired expiry. This keeps both sources of truth: the BloodHound collection and what telemetry has observed since.

4 Projection: provenance to attack edges

The projection is a pure, deterministic function from an OCSF envelope and an identity table to a list of `AttackDeltas`: Assert(edge, event, confidence, ttl), Retract(edge, event), Close(edge, at) and Unresolved(reason). Deltas are keyed by event id, so replaying an envelope is a no-op; a tombstone retracts the support its event contributed; a correction is a retract followed by re-projection.

Design choices. Interactive, remote-interactive and cached logons leave reusable credentials on the host, so they are sessions in the BloodHound sense; network logons (type 3) normally do not, and become `AdminTo` evidence only together with privilege events that the engine does not parse yet. A logoff does not erase credentials from

Table 1: Projection rules v0. lt is the OCSF `logon_type_id`, numbered as the Windows 4624 LogonType [12] for values 2–13.

Event	Condition	Output
Auth. logon	success, $lt \in \{2, 10, 11, 12\}$	`HasSession`($h \to u$), confirmed
Auth. logon	success, $lt = 10$	also `CanRDP`($u \to h$)
Auth. logon	success, $lt \in \{4, 5, 9\}$	`HasSession`, inferred
Auth. logon	$lt \in \{3, 8\}$, failure, no lt	nothing
Auth. logoff	session observed	CLOSE with grace
Process launch	user not system/machine	`HasSession`, inferred, short ttl
Tombstone	—	RETRACT its support

memory immediately, so it closes the session after a grace period rather than retracting it.

Time. The time of an edge is the event time. An out-of-order event can lower *first seen* but never advances expiry, and expiry is driven by the stream watermark, not the wall clock: reprocessing a historical day yields the same state as processing it live.

Identity resolution. Parsers name users `NETBIOS\user` and hosts by whatever the event carries; BloodHound uses UPN-like names, FQDNs and SIDs. The rule uses the SID when present, otherwise normalises against the snapshot's domain table and accepts a short host name only if exactly one FQDN matches. Otherwise it emits UNRESOLVED: a session on the wrong host invents an attack path, which is worse than a missing one.

Defects found. Specifying the rule against the production engine exposed six prerequisites; two were defects. The OCSF projection dropped the logon type, and a failed 4625 parsed by the Sysmon/Security parser reached OCSF without a status—indistinguishable from a successful logon—because that parser flagged failures with a field the projection did not read. Both are fixed upstream with tests; the other four (logoff, privilege and group-change events, SIDs, process host) are missing data, not errors.

5 Incremental distance oracle

For each terminal t (a crown jewel), the oracle keeps $d_t(v)$, the shortest distance from v to t under non-negative weights, and the next hop $\sigma_t(v)$. One structure per terminal answers "how close is any compromised node to t", and distances between terminals give the matrix that routing-style solvers need.

Insertions and decreases propagate the improvement backwards with Dijkstra and touch only nodes that improve. Deletions and increases (Algorithm 1) do nothing unless the edge is in the shortest-path tree; otherwise only the subtree A of nodes whose path went through it is reset, seeded from the unaffected frontier, and repaired.

Algorithm 1: Repair after deleting or raising edge (u, v) for terminal t

```
if σ_t(u) ≠ v then
    return ;                                   // not a tree edge
A ← {u}; W ← deg(u);
foreach x popped from a stack started at u do
    foreach in-neighbour p of x with σ_t(p) = x, p ∉ A
     do
        A ← A ∪ {p}; W ← W + deg(p);
        if W > τ|E| then recompute t from scratch;
         return;
foreach x ∈ A do d_t(x) ← min_{(x,y), y∉A} w(x, y) + d_t(y);
run Dijkstra on the reverse graph restricted to A,
  seeded with those values;
```

Proposition 1. *After Algorithm 1, d_t equals the shortest distances in the updated graph.*

Sketch. A node outside A keeps a tree path to t that avoids (u, v); its length is unchanged and, since the update only removes options, no shorter path appeared. Nodes in A are recomputed by Dijkstra from correct boundary values, which is exact because every shortest path from $x \in A$ leaves A at its first node outside A. □

Work-based fallback. The first version fell back to recomputation when $|A|$ exceeded a fraction of $|V|$. On AD graphs with expanded groups this was the wrong measure: a few hundred nodes can carry hundreds of thousands of edges, and local repair became slower than recomputation. The fallback now counts the in- and out-degree of the nodes added to A and stops as soon as it exceeds $\tau|E|$ ($\tau = 0.1$).

What-if. `what_if(changes)` applies a batch, yields, and restores the previous weights; selection loops that price one candidate rule or trap at a time use it.

6 Joint detection and deception

Detection guarantee. In the *coverage model* [22], a rule r covers the edges whose ATT&CK technique and

telemetry scope it observes, fires with probability q_r and costs c_r expected false positives per day. Under independence, the probability that a path P evades the active rules x is $\prod_{e\in P}\prod_{r\ni e}(1-q_r)^{x_r}$, so requiring every source–terminal path to be detected with probability at least $1-\delta$ is linear in x through potentials:

$$\begin{aligned}&\min\ \textstyle\sum_r c_r x_r\\ &\text{s.t. } \pi_v-\pi_u-\textstyle\sum_{r\ni e} w_r x_r\le 0\quad \forall e=(u,v),\\ &\qquad \pi_s=0,\ \pi_t\ge\theta,\ 0\le\pi\le\theta,\ x\in\{0,1\}^R,\end{aligned}$$

with $w_r = -\log(1-q_r)$ and $\theta = -\log\delta$. The same codebase solves a variant with edge blocking, a randomised triage defender against an informed attacker (Stackelberg), and a lazy constraint-generation version for large graphs.

Traps as rules. The *collector model* [23] places traps on nodes: acquiring a trapped node is detected with certainty. We split every trap candidate v (never the source or a terminal) into $v \to v^{\#}$, redirect v's out-edges to leave from $v^{\#}$, and add a rule $\texttt{trap:}v$ with $q = 1$ that covers only the new edge.

Proposition 2. *For any choice of rules R' and traps T', the minimum evasion weight from s to the terminals in the split instance with rules $R' \cup \{\texttt{trap:}v : v \in T'\}$ equals the one in the original instance with rules R' on edges and traps T' on nodes.*

Sketch. Paths are in bijection: a path that continues from v must traverse $v \to v^{\#}$, which carries exactly the trap weight when $v \in T'$ and zero otherwise; edge weights of redirected edges are unchanged because coverage depends only on technique and scope. Terminals are not split, and a path that ends at v is not a source–terminal path. □

Consequently every coverage solver chooses rules and traps jointly without modification. A trap budget, as in the collector model, is one extra row $\sum_v x_{\texttt{trap:}v} \le k$.

Scope of the equivalence. The coverage attacker wants to reach a terminal, so a trap matters only on transit nodes. The collector gathers value per node under a time budget and a pivot-switch cost; a trap on a dead-end node matters to it and not to this reduction. Evaluating joint solutions against the collector is future work.

6.1 Two attackers, one defender

The two attacker models answer different questions. The coverage attacker wants to reach a crown jewel; the defender guarantees that every path is detected with probability at least $1-\delta$. The collector acquires nodes in sequence under a time budget T, collects value r_v for every acquisition that goes undetected, and stops when detected; the defender minimises the value it can collect. A defense chosen for one attacker need not be good against the other (Section 7.4). We therefore solve

$$\begin{aligned}&\min_B\ \textstyle\sum_s p_s \max_{\pi\in\Pi_s(T)} U_s(B,\pi)\\ &\text{s.t. } |B|\le k,\\ &\qquad \text{every foothold–terminal path detected w.p. } \ge 1-\delta,\end{aligned}$$

where s ranges over footholds with prior p_s and the attacker sees B (the collector model's visible variant). Both constraints are linear in the trap indicators b: the guarantee through the potentials of Section 6 on the split instance (with any fixed set of detection rules as a constant term), and the value through the collector's cuts, one per attacker sequence $\pi = (v_1, \dots, v_m)$ in a growing pool:

$$\theta_s \ \ge\ \textstyle\sum_j c_j\big(1-\sum_{l\le j} b_{v_l}\big), \qquad c_j = r_{v_j}\prod_{l\le j}(1-q_{v_l}).$$

A column-generation loop alternates the master (a MILP over b, θ and π) with the collector's exact best response for each foothold. The master value is a lower bound and the exact response to the incumbent is an upper bound; the loop stops when they meet or no new sequence appears.

To evaluate a coverage defense against the collector we translate it conservatively: traps are kept, and rules raise the detection probability of acquiring v through its *least-watched* in-edge, $q'_v = 1-(1-q_v)\max_{e=(u,v)}\prod_{r\ni e}(1-q_r)$, because the attacker chooses where to come from.

6.2 Rules and traps together

In Section 6.1 detection rules enter only as a fixed set, because their effect on the value-collecting attacker is not linear. With the translation of Section 6.1, surviving the acquisition of v under rules x is

$$s_v(x) = (1-q_v)\exp\Big(-\min_{e=(u,v)}\ \textstyle\sum_{r\ni e} w_r x_r\Big),$$

and the value at position j of a sequence is $c_j(x) = r_{v_j}\prod_{l\le j} s_{v_l}(x)$. The exponent is a maximum of linear functions, hence convex, so $c_j = \exp(\text{convex})$ is convex and every tangent is a valid lower bound: $c_j(x) \ge c_j(\hat{x})\big(1+g_j^{\top}(x-\hat{x})\big)$, with g_j a subgradient of $\log c_j$ at $\hat{x}$ (the rules on each node's least-watched in-edge). Traps zero the suffix of a sequence; we combine both with one variable per position,

$$y_j \ \ge\ c_j(\hat{x})\big(1+g_j^{\top}(x-\hat{x})\big)-M_j\textstyle\sum_{l\le j} b_{v_l},\quad y_j\ge 0,\quad \theta_s\ge\sum_j y_j,$$

with $M_j = c_j(0)$. The master minimises $\lambda_V\sum_s p_s\theta_s + \lambda_A\sum_r c_r x_r$ (loot and alert load), optionally with a loot cap $\sum_s p_s\theta_s \le L$, under the path guarantee and the trap budget. An outer-approximation loop adds, for every attacker sequence in the pool, the tangent at every visited $\hat{x}$, and the attacker's exact best response to the incumbent supplies new sequences and the upper bound. At a visited point the cuts are exact for any trap set (a trap in the prefix makes $y_j = 0$, which is the true value because $c_j \le M_j$), so the loop terminates with matching bounds.

Table 2: Oracle on a synthetic AD graph (`adgen`, scale 2000): 4,583 nodes, 353,526 edges, 4 terminals. Full recomputation for the 4 terminals: 310 ms. Each change is measured and then undone; "fallbacks" counts recomputations from scratch.

Change	mean (ms)	worst (ms)	fallbacks	speed-up
New session	0.025	0.36	0	12,557×
Session expires	0.161	2.27	0	1,924×
AdminTo removed	0.024	0.48	0	12,782×
ACL removed	2.31	226	3	134×
Tree edge near t	20.2	346	21	15×

7 Evaluation

Everything runs in Python on one core of a shared cloud VM; times are single runs and vary between runs by up to ∼1.5× for full recomputation. Code and data live in the Graph Hunter repository [2] under `research/` (private branch).

7.1 Projection

The Python reference and the Rust implementation in the engine pass the same 17 conformance cases (all rules, idempotence, out-of-order arrival, tombstones, corrections, ambiguous hosts and unknown domains); breaking one rule on purpose makes the Rust suite fail. The parser and schema fix adds four tests; 57 tests of the canonical crate and 1,033 of the core engine pass.

7.2 Oracle

Correctness. After every change, distances and paths are compared with a full Dijkstra: 4,800 random changes on 30-node graphs (with and without the fallback), 5×200 on GOADv2 and 150 on a synthetic AD graph with hubs. Disabling the repair of increases on purpose makes the tests fail.

Cost. Table 2 reports the graph whose group expansion creates hubs. Frequent changes cost thousands of times less than recomputation because most of them do not touch any shortest-path tree. The adversarial case, removing a tree edge next to a terminal, averages 15× faster and its worst case (346 ms) stays within 1.1× of recomputation thanks to the work-based fallback; before the fallback was changed, local repair exceeded recomputation. On a sparser random AD-like graph (62,855 nodes, 132,507 edges) the mean repair is 0.016 ms against 199 ms per terminal for recomputation. GOADv2 (46 nodes, 259 edges) is too small to say anything about scale; all its changes take under 0.14 ms.

7.3 Joint detection and deception

Exactness. On 25 random instances (9 nodes, 5 rules, 3 trap candidates) the MILP optimum on the split instance equals a brute force over rules and node traps that does not use the reduction; the cases include optima with traps (8), without (7), and infeasible instances (10). With a budget $k \in \{0, 1, 2\}$ the same holds on 75 more instances. For random choices of rules and traps, evasion distances coincide.

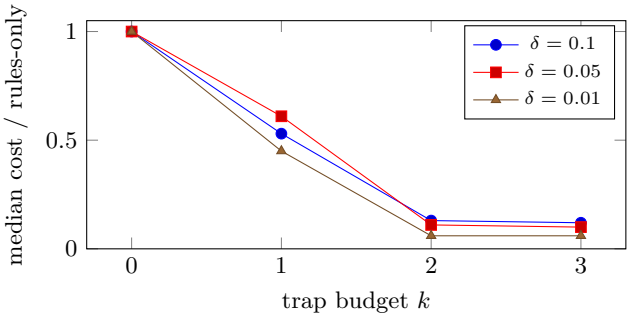


Figure 2: Cost of the guarantee with up to k traps (trap cost 5 FP/day-equivalent), median over the instances feasible with rules only. 20 synthetic instances.

Table 3: Feasible instances out of 20 (trap cost 5).

δ	$k = 0$	$k = 1$	$k = 2$	$k = 3$
0.10	12	17	18	18
0.05	6	14	18	18
0.01	1	7	12	17

Synthetic sweep. On 20 instances of the coverage model's generator (121 nodes, about 340 edges, 25 rules with synthetic detection rates), the best evasion probability reachable with all rules active has median 0.076, so rules alone cannot guarantee $\delta = 0.05$ in 14 of 20 instances (Table 3). Two traps restore feasibility in 18 of 20 and cut the median cost to 11% of the rules-only optimum where both exist (Figure 2); at a trap cost of 15 the reduction is to 25%. Every MILP solves in under 0.2 s.

GOADv2. Using as footholds the users the collector model classifies as unprivileged, the best evasion probability with every rule of the (synthetic) catalog is 0.12 in ESSOS, 0.33 in NORTH and 0.54 in SEVENKINGDOMS. One trap on a transit user or computer gives the $\delta = 0.05$ guarantee in ESSOS and NORTH; in SEVENKINGDOMS the six admissible trap candidates do not suffice. With the coverage importer's default of 20 random footholds, GOAD's small domains include accounts that are already Domain Admins and every instance is infeasible.

7.4 Two attackers on the same graph

Setup. The collector model's synthetic AD generator (scale 60, medium path redundancy; 141 nodes and 557–614 edges) with five footholds per instance and five instances. The coverage instance is built on the same graph:

Table 4: Each defense against both attackers, 5 instances. "Guar.": instances where every foothold–terminal path is detected w.p. ≥ 0.95. V/V_0: loot of the value-collecting attacker relative to no defense (median over instances where the defense exists).

k	Defense	exists	guar.	V/V_0	rules (FP/day)
1	P_k (loot)	5	0	0.68	0
	D_k (rules+traps)	5	5	0.48	19.0
	G_k (combined)	2	2	0.60	0
2	P_k	5	3	0.25	0
	DT_k (guarantee)	5	5	0.50	0
	G_k	5	5	0.44	0
3	P_k	5	1	0.17	0
	DT_k	5	5	0.50	0
	G_k	5	5	0.20	0

same nodes, the collector's footholds as entries, Domain Controllers and Domain Admins as terminals, BloodHound edge types mapped to ATT&CK techniques and telemetry scopes, and a synthetic rule catalog. Traps are allowed on users and computers outside tier 0, never on footholds: a "trap" on a Domain Controller is monitoring of the crown jewel itself and dominates every other choice (on GOADv2 without ADCS edges, one such trap gives the guarantee and cuts the collector's loot from 59.4 to 0.58). The value-collecting attacker has time budget $T = 12$ and no pivot cost; all its best responses are exact. We compare, for a budget of k traps: P_k, the collector model's loot-minimising placement; D_k, the coverage model's cheapest rules plus at most k traps meeting $\delta = 0.05$; DT_k, the same with traps only; and G_k, the combined defender of Section 6.1.

Each single-model optimum fails the other attacker. The collector model's loot-optimal traps give the path guarantee in only 4 of 15 (instance, budget) pairs, and not monotonically: with three traps the optimum gives it in one instance, with two in three (Table 4). Conversely, the coverage model places the fewest traps that meet the guarantee (1.6 on average for $k \geq 2$) and leaves about half the loot, whatever the budget.

The guarantee is cheap once traps can be combined. G_k gives the guarantee whenever it is feasible with traps alone and its loot is within a median 1.00× ($k = 2$) and 1.01× ($k = 3$) of the unconstrained loot optimum; the worst instances cost 2.39× and 1.28×. With a single trap the guarantee is infeasible with traps alone in three of five instances; there the coverage model adds rules (19 false positives/day on average), and in two instances one trap plus one or two rules leaves under 8% of the loot, because rules raise the detection probability of many nodes at once.

Cost. D_k and DT_k take up to 57 s per instance, P_k up to 319 s and G_k up to 291 s, dominated by the exact best responses. Every G_k solution is certified (lower and upper bounds meet). On 14 small instances (scale 20) with every detection rule fixed, G_k matches the enumeration of all trap sets of size at most k.

Table 5: Alert load (median FP/day over the feasible instances, out of 5) needed to meet the path guarantee and keep the loot below αV_0 with at most k traps. Medians are taken over different sets when feasibility differs; per instance, the load never grows with k.

	$k = 0$		$k = 1$		$k = 2$	
α	feas.	FP/day	feas.	FP/day	feas.	FP/day
1.00	4	45.2	5	14.8	5	0.0
0.50	4	45.2	5	35.8	5	0.0
0.25	4	64.1	5	35.8	5	29.3
0.10	4	64.1	5	44.4	5	44.4
0.05	4	74.0	4	64.1	5	67.3

A structural finding on GOADv2. With ADCS edges, an unprivileged user in ESSOS holds an ESC edge straight to the domain. No trap can help (there is no transit node) and the whole rule catalog only reaches an evasion probability of 0.29: one-hop paths from footholds bound every defense that acts on nodes.

Rules and traps together. The joint defender of Section 6.2 minimises the alert load ($\lambda_V = 0$, $\lambda_A = 1$) under the path guarantee $\delta = 0.05$, a budget of k traps and a loot cap $L = \alpha V_0$, on the same five instances (Table 5). Every solution is certified; the slowest took 162 s. On three small instances it matches the enumeration of all rule subsets and trap sets.

Two traps meet the guarantee and halve the loot with no rules in all five instances. Below $\alpha = 0.25$ every solution needs rules, because two traps cannot cover every high-value node; at $\alpha = 0.1$, two traps cut the alert load of the rules-only solution by 5–47% (median 23%) on the four instances where rules alone are feasible. The guarantee alone already bounds the loot in some instances: the cheapest rules-only defense leaves about 5% of the loot in two instances and about half in two others, so whether a loot cap costs anything depends on where value sits relative to the monitored edges. Since the rules' effect on this attacker goes through the least-watched in-edge translation, these loads are upper bounds for an attacker who cannot pick its entry edge.

8 Discussion and limitations

- **Traps are perfect by assumption.** The reduction inherits the collector model's $q = 1$; one trap then dominates any number of imperfect rules. A realistic trap model needs an interaction probability, which the reduction already supports through the rule's q.

- **Translation between models.** A coverage rule acts on edges and the collector sees per-node detection; we take the least-watched in-edge, which favours the attacker. A joint model where the value-collecting attacker also chooses edges would remove this step.
- **Synthetic detection content.** Catalogs and detection rates are synthetic; the coverage model already imports Sigma rules, which is the next step.
- **Independence.** The linear guarantee assumes independent firings; the coverage model quantifies the effect of common-cause correlation separately.
- **No large real AD.** GOADv2 is small; client collections cannot enter the repository. The worst case of the oracle on real hubs, and the cross-evaluation, are measured only on synthetic generators.
- **Projection parameters.** Session lifetimes and grace periods are uncalibrated defaults. Of four missing-data prerequisites, logoff, SIDs and the process host are now carried by the parsers (under review); privilege and group-change events remain open.

9 Roadmap

(1) Connect the Rust projection to the ingestion journal so that G_t is maintained live, and close the missing-data prerequisites. (2) Remove the least-watched translation by letting the value-collecting attacker choose edges, so that the joint defender optimises against one attacker that both models agree on. (3) Unify the certificate formats of the solvers. (4) Calibrate edge weights and detection rates from attack chains reconstructed by the engine's Steiner-arborescence module and from emulation runs. (5) Measure everything on a large real collection under the appropriate data agreements.

Items to verify before submission

- OCSF 1.4 `logon_type_id` values: checked against a LinkML mirror of OCSF and Microsoft's enumeration, not against the official schema site (not reachable from our environment).
- Page range of [8]: the two source bibliographies disagree (305–312 vs 327–334).

References

[1] Ahmed H. Anwar, Charles Kamhoua, and Nandi Leslie. Honeypot allocation over attack graphs in cyber deception games. In *2020 International Conference on Computing, Networking and Communications (ICNC)*, pages 502–506. IEEE, 2020.

[2] BASE4 Security. Graph Hunter: Graph-based and hypothesis-driven threat hunting. https://github.com/Base4Security/GraphHunter, 2026.

[3] Camil Demetrescu and Giuseppe F. Italiano. A new approach to dynamic all pairs shortest paths. *Journal of the ACM*, 51(6):968–992, 2004.

[4] Diksha Goel, Max Ward, Aneta Neumann, Frank Neumann, Hung Nguyen, and Mingyu Guo. Defending Active Directory by combining neural network based dynamic program and evolutionary diversity optimisation. In J. E. Fieldsend and M. Wagner, editors, *Proceedings of the Genetic and Evolutionary Computation Conference (GECCO '22)*, pages 1191–1199, New York, NY, 2022. ACM.

[5] Mingyu Guo, Jialiang Li, Aneta Neumann, Frank Neumann, and Hung Nguyen. Practical fixed-parameter algorithms for defending Active Directory style attack graphs. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 36, pages 9360–9367, 2022.

[6] Mingyu Guo, Max Ward, Aneta Neumann, Frank Neumann, and Hung Nguyen. Scalable edge blocking algorithms for defending Active Directory style attack graphs. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 37, pages 5649–5656, 2023.

[7] Eitan Israeli and R. Kevin Wood. Shortest-path network interdiction. *Networks*, 40(2):97–111, 2002.

[8] Manish Jain, Dmytro Korzhyk, Ondřej Vaněk, Vincent Conitzer, Michal Pěchouček, and Milind Tambe. A double oracle algorithm for zero-sum security games on graphs. In *Proceedings of the 10th International Conference on Autonomous Agents and Multiagent Systems (AAMAS 2011)*, pages 327–334, 2011.

[9] Somesh Jha, Oleg Sheyner, and Jeannette M. Wing. Two formal analyses of attack graphs. In *Proceedings of the 15th IEEE Computer Security Foundations Workshop (CSFW)*, pages 49–63, 2002.

[10] Sven Koenig, Maxim Likhachev, and David Furcy. Lifelong planning A*. *Artificial Intelligence*, 155(1–2):93–146, 2004.

[11] m4lwhere. Bloodhound-CE-Sample-Data: Sample data to load into BloodHound Community Edition, based on GOADv2. https://github.com/m4lwhere/Bloodhound-CE-Sample-Data. SharpHound v2.3.3 collection. Accessed 2026-10-22.

[12] Microsoft. SECURITY_LOGON_TYPE enumeration (ntsecapi.h). https://learn.microsoft.com/windows/win32/api/ntsecapi/ne-ntsecapi-security_logon_type.

[13] David P. Morton, Feng Pan, and Kevin J. Saeger. Models for nuclear smuggling interdiction. *IIE Transactions*, 39(1):3–14, 2007.

[14] Huy Q. Ngo, Mingyu Guo, and Hung Nguyen. Catch me if you can: Effective honeypot placement in dynamic AD attack graphs. In *IEEE INFOCOM 2024 – IEEE Conference on Computer Communications*, pages 451–460. IEEE, 2024.

[15] Di H. Nguyen, Yongjia Song, and J. Cole Smith. A two-stage network interdiction-monitoring game. *Networks*, 81(3):334–358, 2023.

[16] Steven Noel and Sushil Jajodia. Optimal IDS sensor placement and alert prioritization using attack graphs. *Journal of Network and Systems Management*, 2008.

[17] OCSF Project. Open cybersecurity schema framework, version 1.4. https://schema.ocsf.io.

[18] Orange Cyberdefense. GOAD: Game of Active Directory. https://github.com/Orange-Cyberdefense/GOAD. Accessed 2026-10-22.

[19] Jeffrey Pawlick, Edward Colbert, and Quanyan Zhu. A game-theoretic taxonomy and survey of defensive deception for cybersecurity and privacy. *ACM Computing Surveys*, 52(4):1–28, 2019.

[20] G. Ramalingam and Thomas Reps. An incremental algorithm for a generalization of the shortest-path problem. *Journal of Algorithms*, 21(2):267–305, 1996.

[21] Aaron Schlenker et al. Don't bury your head in warnings: A game-theoretic approach for intelligent allocation of cyber-security alerts. In *Proceedings of IJCAI*, pages 381–387, 2017.

[22] Lucas Sotomayor. detcover: Minimum-cost detection cuts on attack graphs, 2026. Unpublished code and manuscript, commit 70b6a13.

[23] Lucas Sotomayor. PERIPLO: Certified randomized deception against multi-target attackers in Active Directory attack graphs. https://github.com/sotomayorlucas/periplo, 2026. Code and manuscript, commit 1456ec1.

[24] SpecterOps. BloodHound: Six Degrees of Domain Admin. https://github.com/SpecterOps/BloodHound. Accessed 2026-10-22.

[25] Apurva Virkud, Muhammad Adil Inam, Andy Riddle, Jason Liu, Gang Wang, and Adam Bates. How does endpoint detection use the MITRE ATT&CK framework? In *Proceedings of the 33rd USENIX Security Symposium*, pages 3891–3908, 2024.

[26] Lingyu Wang, Steven Noel, and Sushil Jajodia. Minimum-cost network hardening using attack graphs. *Computer Communications*, 2006.